

Enterprise Cloud-Native DevOps Platform

Project summary:

This project demonstrates an enterprise-style cloud-native application built with a microservices architecture. It includes a React frontend, a Java Spring Boot product service, and a Python Flask order service, all designed to run in a containerized and orchestrated environment.

Architecture overview:

Source code is managed in GitHub. Docker packages each application into portable containers. Docker Hub stores container images. Terraform handles infrastructure as code. Kubernetes orchestrates the runtime environment. Azure DevOps automates the deployment pipeline.

Technology stack:

- Frontend: React.
- Backend services: Java Spring Boot and Python Flask.
- Containerization: Docker and Docker Compose.
- Registry: Docker Hub
- Orchestration: Kubernetes.
- Infrastructure as Code: Terraform.
- CI/CD: Azure DevOps.
- Version control: GitHub.

Application development:

The frontend provides the user interface for browsing and interacting with the system. The product service exposes REST endpoints for product operations, and the order service manages order-related requests. This separation into services makes the system modular and easier to scale.

Docker and containerization:

Each service is packaged into its own Docker image to ensure consistent behavior across environments. Docker Compose supports local multi-container execution and internal networking between services. This approach reduces environment issues and improves portability.

Kubernetes orchestration:

Kubernetes manages deployment, scaling, and service discovery for the application. Pods run the application containers, deployments manage replicas and updates, and services provide stable internal communication. The frontend can be exposed externally, while backend services stay private inside the cluster.

Terraform automation:

Terraform is used to provision and manage infrastructure in a repeatable way. The standard workflow is `terraform init`, `terraform plan`, and `terraform apply`. This reduces manual configuration and supports consistent deployments.

Azure DevOps pipeline:

Azure DevOps is used for deployment automation instead of Jenkins. It detects changes from GitHub, triggers the release workflow, pulls the required container images, and deploys the application to Kubernetes. This provides a modern enterprise deployment path with faster releases and reduced downtime.

Monitoring and future enhancements:

The project can be extended with Prometheus and Grafana for monitoring, Kubernetes Secrets for improved security, and Horizontal Pod Autoscaler for better scaling. Additional enhancements could include Helm charts, ArgoCD, and SonarQube.

Professional skills demonstrated:

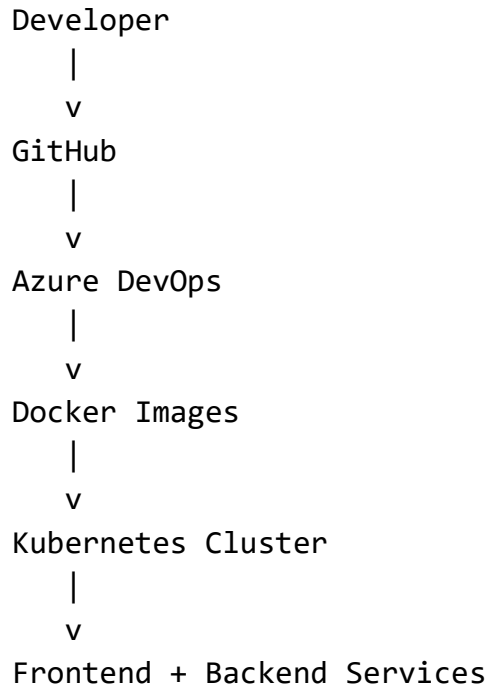
This project demonstrates microservices design, REST API development, Docker containerization, Kubernetes orchestration, Infrastructure as Code, CI/CD automation, cloud-native deployment, and GitHub-based source control workflows.

Conclusion:

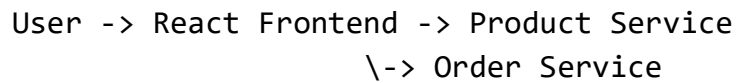
This project is a complete enterprise DevOps workflow that shows how modern applications are developed, containerized, deployed, and managed. It is suitable for presenting to management because it highlights both technical depth and practical business value.

Diagrams to include

Architecture flow



Service interaction



Deployment lifecycle

